

Gestor Financiero

Documentación Técnica del Proyecto

Equipo de Desarrollo

Mayo 2026

Contents

1 Documentación técnica — Gestor Financiero	1
1.1 1. Resumen ejecutivo	1
1.2 2. Arquitectura general	2
1.3 3. Stack tecnológico (qué + por qué)	3
1.4 4. Modelo de datos	5
1.5 5. Backend en detalle	7
1.6 6. Frontend en detalle	9
1.7 7. Seguridad	10
1.8 8. Reportes y PDFs	12
1.9 9. Sistema de emails (Gmail SMTP)	12
1.10 10. Configuración local	13
1.11 11. Configuración del servidor MonsterASP	14
1.12 12. CI/CD con GitHub Actions	15
1.13 13. Posibles preguntas del docente	16
1.14 14. Limitaciones conocidas	18
1.15 15. Resumen ejecutivo en una página	18

1 Documentación técnica — Gestor Financiero

Documento de referencia para la exposición del proyecto. Está pensado para que el equipo pueda responder cualquier pregunta del docente sobre arquitectura, tecnologías, deploy, base de datos, seguridad y configuración.

1.1 1. Resumen ejecutivo

Gestor Financiero es una aplicación web para la gestión de finanzas personales. Permite a cada usuario llevar el control de sus ingresos y gastos, planificar presupuestos mensuales, registrar y amortizar deudas, generar reportes imprimibles y exportar a PDF. El sistema está pensado para uso individual: cada usuario solo ve y manipula sus propios datos.

URL en producción: <https://gestor-financiero.runasp.net> Hosting: MonsterASP.NET (plan gratuito) Repositorio: GitHub con CI/CD vía GitHub Actions.

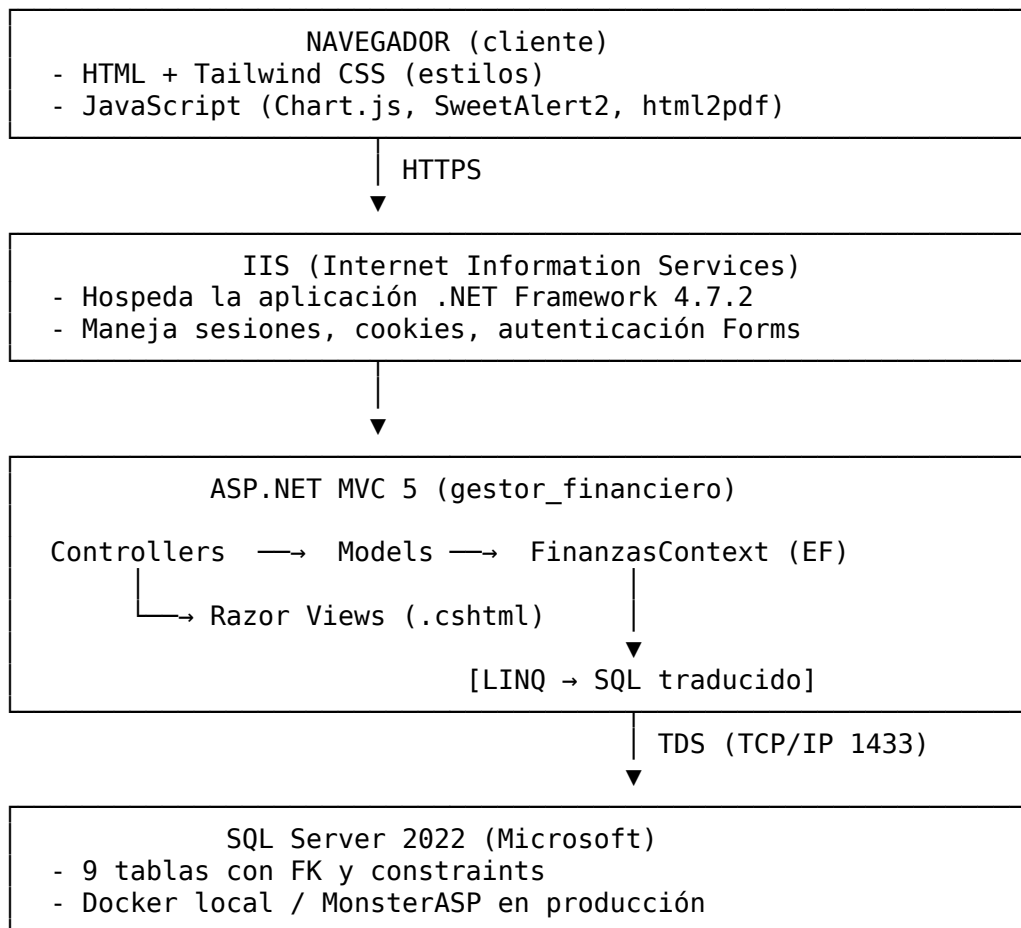
1.2 2. Arquitectura general

1.2.1 Estilo arquitectónico

Es una aplicación **monolítica** con arquitectura **MVC (Modelo-Vista-Controlador)** clásica de ASP.NET. Se eligió monolito porque:

- El alcance es acotado (un solo dominio: finanzas personales).
- Un solo equipo desarrollándolo y manteniéndolo.
- Hosting gratuito que no soporta arquitecturas distribuidas.
- Despliegue y operación más simples.

1.2.2 Diagrama de capas



1.2.3 Componentes externos

Componente	Función	Producto usado
Envío de emails	OTP de registro y reset de contraseña	Gmail SMTP (smtp.gmail.com:587 TLS)
CI/CD	Build automático + deploy en cada push	GitHub Actions
Transporte deploy	Subir archivos al servidor	FTP (acción SamKirkland/FTP-Deploy-Action)

Componente	Función	Producto usado
Generación PDF	Exportar reportes	Cliente (html2pdf.js corre en el navegador)

1.3 3. Stack tecnológico (qué + por qué)

1.3.1 Backend

Tecnología	Versión	Por qué
ASP.NET MVC 5	5.2.9	Patrón MVC nativo de Microsoft, maduro y bien documentado. Compatible con hosting compartido económico (no requiere .NET Core ni containers en el server).
.NET Framework	4.7.2	Es lo que soporta gratis el plan free de MonsterASP. Si hubiéramos usado .NET 6/7/8, el hosting gratuito no lo aceptaba.
Entity Framework 6	6.4.4	ORM oficial de Microsoft. Permite escribir consultas con LINQ en lugar de SQL crudo, hace tracking de cambios y maneja transacciones automáticamente.
Razor	3.2.9	Motor de plantillas de Microsoft. Permite mezclar C# con HTML de forma muy expresiva.
System.Web.Helpers (PBKDF2)	—	Para hashear contraseñas con Crypto.HashPassword / VerifyHashedPassword. Usa PBKDF2 con 1000 iteraciones + salt aleatorio de 16 bytes.
Forms Authentication	—	Sistema de autenticación basado en cookies firmadas con HMAC. Estándar en MVC.

1.3.2 Frontend

Tecnología	Versión	Por qué
Tailwind CSS	3.4.17	Utility-first CSS. Permite diseñar rápido sin escribir CSS custom para cada componente. Compilado localmente (no usa CDN), genera un <code>app.css</code> minificado de ~25 KB con solo las clases usadas.
Chart.js	4.4.7	Gráficos del dashboard y reportes (barras, doughnut, pie). API simple, buena performance.
SweetAlert2	11.14.5	Alertas modernas tipo “toast” para feedback de éxito/error. Reemplaza al <code>alert()</code> feo del navegador.
html2pdf.js	0.10.2	Combina jsPDF + html2canvas. Permite “fotografiar” cualquier vista HTML y descargarla como PDF directamente desde el navegador, sin tocar el servidor.
Lucide Icons	0.469	Set de íconos vectoriales modernos. Servidos como fuente local (woff2), no como CDN.
Inter font	5.1.1	Tipografía moderna y legible. Cargada localmente con <code>@fontsource/inter</code> .

1.3.3 Base de datos

Tecnología	Versión	Por qué
SQL Server	2022	Compatible con Entity Framework. Soporta constraints, transacciones, tipos DECIMAL para dinero (preciso, no usa FLOAT). Hay versión gratuita en MonsterASP.
Docker + Docker Compose	24+	Para correr SQL Server localmente sin instalarlo en Windows. Un solo comando (<code>.\start.ps1</code>) levanta toda la BD ya configurada.

1.3.4 DevOps / Deploy

Herramienta	Por qué
GitHub Actions	CI/CD gratis e integrado con el repo. Build automático en cada push.
MSBuild	Compila proyectos .NET Framework (Roslyn no alcanza para web projects).
MonsterASP.NET	Hosting Windows + IIS + SQL Server gratis (plan free). Permite .NET Framework 4.7.2.
FTP-Deploy-Action	Sube el publish/ por FTP al hosting.
sqlcmd	Cliente de línea de comandos de SQL Server. Lo usamos para aplicar migraciones automáticamente en el deploy.

1.3.5 ¿Por qué no usamos algunas tecnologías populares?

Tecnología	Por qué NO la usamos
.NET Core / .NET 8	El plan free de MonsterASP solo soporta .NET Framework. Migrar implica pagar hosting.
Crystal Reports	Requiere instalar un runtime nativo (~150 MB de DLLs C++) en el servidor, lo cual no está permitido en hosting compartido gratuito. Además es tecnología en declive.
AJAX / fetch	No fue necesario para este alcance: los forms con submit clásico funcionan bien. El único cálculo dinámico (registrar pago de deuda) se hace 100% en el cliente con JS local.
React / Vue / Angular	El alcance es chico, la velocidad de Razor con servidor renderizando es más que suficiente. SPA habría sido sobreingeniería.
NoSQL (Mongo, etc.)	Los datos son altamente relacionales (usuario → presupuestos → detalles → categorías). SQL es el ajuste natural.

1.4 4. Modelo de datos

1.4.1 Diagrama entidad-relación

Disponible en database/diagrama-er.svg y diagrama-er.png. Lo siguiente es la estructura textual:

Usuario (id_usuario PK, nombre, email UNIQUE, password_hash, fecha_registro, activo)

├─ Presupuesto (id_presupuesto PK, id_usuario FK, anio, mes)
 └─ UNIQUE(id_usuario, anio, mes)

├─ PresupuestoDetalle (id_detalle PK, id_presupuesto FK, id_categoria FK, estimado, real)

├─ Transaccion (id_transaccion PK, id_usuario FK, id_categoria FK, monto, fecha, notas)

├─ Deuda (id_deuda PK, id_usuario FK, nombre, saldo_actual, tasa_interes, pago_minimo, notas)

├─ PagoDeuda (id_pago PK, id_deuda FK, fecha, pago_minimo, pago_extra, interes, capital,

└─ ResumenAnual (id_resumen PK, id_usuario FK, anio, ingresos, ahorros, gastos_fijos, gastos_

└ PasswordResetToken (id_token PK, id_usuario FK, token, proposito, expiracion, usado, cread

Categoria (id_categoria PK, nombre, tipo) ← catálogo global (compartido)

└ PresupuestoDetalle
└ Transaccion

_AppliedMigrations (nombre PK, aplicada_en) ← tabla técnica de tracking de migraciones

1.4.2 Detalle de cada tabla

1.4.2.1 Usuario La entidad central. Cada usuario tiene su propio universo de datos. - password_hash: PBKDF2 con salt aleatorio (formato Base64). - email: UNIQUE para evitar duplicados. - activo: BIT que arranca en 0 al registrarse. Solo se vuelve 1 después de verificar el OTP enviado por email.

1.4.2.2 Categoria Catálogo **global** (no por usuario). Tiene un campo tipo con valores: - INGRESO (verde) - AHORRO (azul) - GASTO_FIJO (ámbar) - GASTO_VARIABLE (rojo) - DEUDA (violeta)

Se semillea automáticamente con 10 categorías comunes al crear la BD.

1.4.2.3 Presupuesto y PresupuestoDetalle Un **Presupuesto** representa una planificación mensual de un usuario. Su **detalle** son las categorías estimadas vs reales. La columna real está protegida en EF con [Column("real")] porque real es palabra reservada de T-SQL.

1.4.2.4 Transaccion Movimientos individuales de dinero. Cada uno se asocia a una categoría que determina si es ingreso, gasto u otro.

1.4.2.5 Deuda y PagoDeuda Una **Deuda** tiene saldo actual, tasa de interés y pago mínimo. Cada **PagoDeuda** desglosa: cuánto fue mínimo, cuánto extra, cuánto a interés y cuánto a capital. El saldo restante se calcula automáticamente.

1.4.2.6 ResumenAnual Resumen agregado de un año (ingresos, ahorros, gastos fijos/variables, deudas). Sirve para los gráficos comparativos.

1.4.2.7 PasswordResetToken Tabla de códigos OTP. Tiene un campo proposito que distingue: - REGISTRO → para activar una cuenta nueva. - RESET_PASSWORD → para recuperar una contraseña olvidada.

1.4.2.8 _AppliedMigrations Tabla técnica que el sistema de migraciones usa para saber qué scripts SQL ya se aplicaron, así no los reejecuta.

1.4.3 Integridad referencial

Todas las FK están declaradas en SQL con CONSTRAINT FK_X_Y. Entity Framework las respeta y EF Code-First las “descubre” por las propiedades de navegación (virtual Usuario { get; set; }, etc).

1.4.4 Índices

Sobre las columnas más consultadas: - IX_Presupuesto_Usuario - IX_PresupuestoDetalle_Presupuesto - IX_Transaccion_Usuario_Fecha (compuesto) - IX_Deuda_Usuario - IX_PagoDeuda_Deuda - IX_PwdResetToken_Usuario

Mejoran los WHERE id_usuario = X y los JOIN.

1.4.5 Tipos de datos importantes

- **Dinero:** DECIMAL(12,2) — preciso para 10 dígitos enteros + 2 decimales. NUNCA FLOAT (pierde precisión en sumas).
- **Tasas:** DECIMAL(5,2) — hasta 999.99%.
- **Fechas de transacción:** DATE (sin hora).
- **Auditoría:** DATETIME (con hora, en UTC).

1.5 5. Backend en detalle

1.5.1 Estructura del proyecto

```

gestorFinanciero/
├── App_Start/
│   ├── DataSeeder.cs           ← inserta usuario demo si no existe
│   ├── FilterConfig.cs        ← agrega filtro global [Authorize]
│   ├── RouteConfig.cs         ← rutas {controller}/{action}/{id}
│   ├── CategoriaHelper.cs     ← construye dropdowns agrupados por tipo
│   └── EmailService.cs        ← envío SMTP de Gmail + plantillas HTML
├── Controllers/
│   ├── BaseAuthController.cs  ← clase base con propiedad CurrentUserId
│   ├── AccountController.cs   ← Login, Register, OTP, ResetPassword, ChangePassword, Logout
│   ├── HomeController.cs      ← Dashboard con KPIs y gráficos
│   ├── UsuarioController.cs   ← "Mi perfil"
│   ├── CategoriaController.cs ← CRUD del catálogo global
│   ├── PresupuestoController.cs ← CRUD de presupuestos del usuario
│   ├── PresupuestoDetalleController.cs
│   ├── TransaccionController.cs
│   ├── DeudaController.cs
│   ├── PagoDeudaController.cs
│   ├── ResumenAnualController.cs
│   └── ReportesController.cs  ← 4 reportes filtrables
├── Models/                    ← clases POCO + FinanzasContext (DbContext)
├── ViewModels/                ← VMs para forms (no son entidades de BD)
├── Views/                     ← .cshtml organizados por controller
├── Content/                   ← CSS compilado, fuentes
├── Scripts/                   ← JS de Chart.js, SweetAlert2, html2pdf
├── database/                  ← scripts SQL + diagrama ER
├── .github/workflows/         ← CI/CD
├── docker-compose.yml         ← define el contenedor SQL Server local
├── Web.config                 ← configuración local (BD apuntando a Docker)
├── Web.Release.config         ← transformación XML para producción
└── package.json               ← dependencias npm (Tailwind, Chart.js, etc.)

```

1.5.2 Patrón MVC en este proyecto

Modelo: Models/*.cs — clases POCO con DataAnnotations ([Required], [StringLength], [Column]). EF las mapea a tablas SQL.

Vista: Views/<Controller>/*.cshtml — Razor renderiza HTML usando el modelo que le pasa el controller. Usa Tailwind para estilos.

Controlador: recibe requests HTTP, valida, consulta a EF, devuelve una vista o redirige.

1.5.3 Cómo se conecta el backend a la base

```
Web.config
<connectionStrings>
  <add name="FinanzasContext"
        connectionString="Server=...;..."
        providerName="System.Data.SqlClient"/>
```

(EF lee el connection string por nombre)

```
Models/FinanzasContext.cs

public class FinanzasContext : DbContext {
  public FinanzasContext()
    : base("name=FinanzasContext") { }

  public DbSet<Usuario> Usuarios { get; set; }
  public DbSet<Presupuesto> Presupuestos { ... }
  ... (uno por tabla)
}
```

(cada controller hace: var db = new FinanzasContext())

```
Controller
var lista = await db.Transacciones
    .Include(t => t.Categoria)
    .Where(t => t.IdUsuario == CurrentUserId)
    .ToListAsync();
```

Database.SetInitializer<FinanzasContext>(null) en el constructor del context **desactiva** las migraciones automáticas de EF. Usamos scripts SQL manuales en su lugar (más control).

1.5.4 Cómo se asegura que cada usuario solo vea sus datos

BaseAuthController expone CurrentUserId que lee User.Identity.Name (que se setea al login con FormsAuthentication.SetAuthCookie(usuario.IdUsuario.ToString(), ...)).

Todos los controllers heredan de BaseAuthController y filtran las queries:

```
public async Task<ActionResult> Index()
{
  var transacciones = await db.Transacciones
    .Where(t => t.IdUsuario == CurrentUserId)  ← clave
    .ToListAsync();
  return View(transacciones);
}
```

Para acciones que modifican (Edit/Delete), siempre verificamos ownership:

```
var entity = await db.X.FirstOrDefaultAsync(
  x => x.Id == id && x.IdUsuario == CurrentUserId);
if (entity == null) return HttpNotFound(); // 404 si no es del usuario
```

Esto previene ataques **IDOR** (Insecure Direct Object Reference): aunque alguien manipule la URL /Transaccion/Edit/123 con un ID de otro usuario, recibe 404.

1.5.5 Uso de LINQ

Es masivo. Cada controlador hace consultas como:

```
var resumen = db.Transacciones
    .Include(t => t.Categoria)
    .Where(t => t.IdUsuario == userId && t.Fecha >= inicio)
    .GroupBy(t => t.Categoria.Tipo)
    .Select(g => new { Tipo = g.Key, Total = g.Sum(x => x.Monto) })
    .ToList();
```

EF lo traduce a SQL parametrizado (inmune a SQL injection):

```
SELECT Categoria.Tipo, SUM(Transaccion.Monto)
FROM Transaccion
INNER JOIN Categoria ON Transaccion.id_categoria = Categoria.id_categoria
WHERE Transaccion.id_usuario = @userId
AND Transaccion.fecha >= @inicio
GROUP BY Categoria.Tipo;
```

1.6 6. Frontend en detalle

1.6.1 Layout principal (Views/Shared/_Layout.cshtml)

- **Sidebar** fijo a la izquierda con navegación principal.
- **Header** arriba con título de la página y botón “Nueva transacción”.
- **Main content** centrado, max-width responsivo.
- En mobile (< 768px): el sidebar se vuelve un **drawer** (cajón) que se abre con un botón hamburguesa.

1.6.2 Razor: cómo se renderiza una vista

1. Usuario navega a /Transaccion/Index
2. RouteConfig matchea: controller=Transaccion, action=Index
3. Controller carga datos con LINQ → devuelve `return View(model)`
4. ASP.NET MVC busca: Views/Transaccion/Index.cshtml
5. Razor combina el .cshtml con _ViewStart.cshtml y _Layout.cshtml
6. Resultado: HTML completo con sidebar + tabla de transacciones

1.6.3 Tailwind CSS

- Se compila con `npm run build` que ejecuta `tailwindcss -i Content/input.css -o Content/css/app.css --minify`.
- Escanea todos los .cshtml y genera **solo** las clases utilizadas.
- El CSS final pesa ~25 KB (vs ~3 MB del CDN play que es el desarrollo).
- Si un compañero agrega una clase nueva en una vista, hay que volver a correr `npm run build` para que aparezca en el CSS.

1.6.4 Chart.js

Se usa en: - Views/Home/Index.cshtml — dashboard con gráfico de barras (ingresos vs gastos) y doughnut (gastos por categoría). - Views/ResumenAnual/Index.cshtml — gráfico de barras anual + pie de distribución.

Los datos se pasan al JS serializándolos con `Html.Raw(Json.Encode(modelo))`.

1.6.5 SweetAlert2 — sistema de notificaciones

En el `_Layout.cshtml` hay un bloque global:

```
@{
    var swalSuccess = TempData["Success"] as string;
    var swalError    = TempData["Error"] as string;
}
@if (swalSuccess != null) {
    <script>Swal.fire({ toast: true, position: 'top-end', icon: 'success', title: '@swalSuccess',
}
```

Cualquier controller que haga `TempData["Success"] = "Mensaje"` antes de un `RedirectToAction(...)` dispara un toast en la próxima página. Patrón “post-redirect-toast”.

1.6.6 Generación de PDFs (html2pdf)

Pantallas con botón “Descargar PDF”: - Dashboard (Home/Index) - 4 reportes (Transacciones, Deudas, Presupuesto, ResumenAnual) - Detalle de presupuesto - Detalle de deuda - Resumen anual

Cómo funciona:

1. El usuario hace clic en "Descargar PDF"
2. `html2canvas` "fotografía" el `<div id="reporte-pdf">`
3. Esa imagen PNG se mete en un PDF (A4) con `jsPDF`
4. El navegador descarga el archivo: "Reporte_Transacciones_2026-05-29.pdf"

Ventaja: **cero código en el servidor**. La generación es 100% client-side. No requiere instalar Crystal Reports ni wkhtmltopdf en el hosting (que es un plan compartido gratuito que no permite binarios nativos).

1.7 7. Seguridad

1.7.1 Autenticación

Forms Authentication de ASP.NET:

```
// Al loguear correctamente:
FormsAuthentication.SetAuthCookie(usuario.IdUsuario.ToString(), rememberMe);
```

Esto setea una cookie firmada con HMAC usando la `<machineKey>` del `Web.config`. La cookie contiene el `IdUsuario`. En las siguientes requests, el filtro `[Authorize]` lee esa cookie y la verifica.

`<machineKey>` está **fija** en `Web.Release.config` (no autogenerada) para que sobreviva los reinicios del app pool de MonsterASP. Si fuera autogenerada, cada reinicio invalidaría todas las cookies activas.

1.7.2 Autorización

`FilterConfig.cs` agrega `[Authorize]` como filtro **global**: todas las rutas requieren login, excepto las explícitamente marcadas con `[AllowAnonymous]` (Login, Register, ForgotPassword, ResetPassword, VerifyOtp).

1.7.3 Contraseñas (PBKDF2)

```
// Guardar:
usuario.PasswordHash = Crypto.HashPassword(model.Password);
```

```
// Verificar:  
if (Crypto.VerifyHashedPassword(usuario.PasswordHash, model.Password)) { ... }
```

System.Web.Helpers.Crypto: - Algoritmo: **PBKDF2** con HMAC-SHA1. - Iteraciones: **1000**. - Salt: aleatorio de **16 bytes** generado por hash. - Output: Base64 de 60 caracteres aproximadamente. - Resistente a ataques de fuerza bruta y rainbow tables.

1.7.4 CSRF (anti-forgery)

Todos los forms POST tienen `@Html.AntiForgeryToken()` y los controllers tienen `[ValidateAntiForgeryToken]`. Si alguien envía un POST sin el token correcto, ASP.NET rechaza con 403.

1.7.5 Verificación de cuenta con OTP

Cuando un usuario se registra: 1. Se crea con `activo = false`. 2. Se genera un código OTP de 6 dígitos numéricos. 3. Se envía por email vía SMTP de Gmail. 4. El usuario lo ingresa en `/Account/VerifyOtp`. 5. Si el código es válido y no expiró (10 min) $\rightarrow$ `activo = true`. 6. Si intenta loguearse antes de verificar, el sistema le manda otro OTP automáticamente.

1.7.6 Recuperación de contraseña

1. Usuario en Login $\rightarrow$ “¿Olvidaste tu contraseña?”.
2. Ingresa email $\rightarrow$ recibe OTP por correo.
3. Lo valida en `/Account/VerifyOtp`.
4. Le aparece la pantalla de “Nueva contraseña”.
5. Guarda. Login con la nueva clave.

1.7.7 Cambio de contraseña desde el perfil

`/Account/ChangePassword` (requiere `[Authorize]`): - Pide contraseña actual (valida con `Crypto.VerifyHashedPassword`). - Pide nueva contraseña (mínimo 8 caracteres). - Pide confirmación (con `[Compare]`). - Actualiza.

No requiere OTP porque el usuario ya demostró su identidad (está logueado y conoce la contraseña actual).

1.7.8 Prevención de IDOR

Como se explicó arriba, todas las queries filtran por `CurrentUserId`. Aun manipulando URLs, un usuario solo accede a sus propios datos.

1.7.9 Email enumeration protection

Cuando se pide “Olvidé mi contraseña”: - Si el email existe $\rightarrow$ manda OTP. - Si NO existe $\rightarrow$ no hace nada. - **En ambos casos**, el mensaje en pantalla es el mismo: *“Si el correo está registrado, te enviamos un código...”*.

Así un atacante no puede usar el endpoint para descubrir qué emails están en la base.

1.7.10 Headers de seguridad (en producción)

Web.Release.config agrega: - `X-Content-Type-Options: nosniff` — el navegador no debe adivinar el tipo MIME. - `X-Frame-Options: SAMEORIGIN` — previene clickjacking. - `Referrer-Policy: strict-origin-when-cross-origin` — no filtra la URL completa al ir a otros sitios.

1.8 8. Reportes y PDFs

1.8.1 Reportes disponibles

Reporte	Ruta	Contenido
Transacciones	/Reportes/Transacciones?desde={id}&hasta={id}	Detalle de transacciones + totales (ingresos, gastos, balance).
Deudas	/Reportes/Deudas	Saldo, tasa, pagos totales por deuda.
Presupuesto	/Reportes/Presupuesto/{id}	Detalle de un presupuesto con barras de progreso.
Resumen Anual	/Reportes/ResumenAnual	Comparativa multi-año con gráficos.

1.8.2 Dos formas de exportar

Imprimir / “Guardar como PDF” del navegador (servidor): - Cada vista tiene CSS con `@media print` que oculta el sidebar. - Botón “Imprimir” llama a `window.print()`. - El usuario elige “Guardar como PDF” en el diálogo del navegador. - Resultado: PDF con texto seleccionable, alta calidad.

Descargar PDF (cliente, automático): - Botón “Descargar PDF” verde. - Captura el `<div id="reporte-pdf">` con `html2canvas` → lo mete en `jsPDF`. - Descarga directa con nombre como `Reporte_Transacciones_2026-01-01_2026-05-29.pdf`. - Resultado: PDF con la vista exacta (gradientes, gráficos, todo).

1.8.3 Por qué NO usamos Crystal Reports

(Pregunta típica del docente, conviene tener la respuesta clara)

1. **MonsterASP free** no permite instalar el runtime nativo de Crystal Reports (DLLs C++ que requieren acceso al GAC y `system32`, bloqueado en hosting compartido).
2. Crystal Reports fue diseñado para **WebForms** (.aspx), no para MVC. Integrarlo en MVC requiere “tricks” que rompen el patrón.
3. Es **tecnología en declive** (de SAP, sin soporte para .NET Core/5+).
4. Habría que **rediseñar cada reporte** en el Crystal Reports Designer (UI WYSIWYG separada), descartando todo el trabajo de las vistas Razor + Tailwind.
5. Tiene **licencia confusa** (registro obligatorio, versiones específicas por VS).

En su lugar usamos **html2pdf.js** que: - Es gratis (MIT). - Funciona 100% en el navegador, sin tocar el server. - Aprovecha las vistas Razor que ya teníamos hechas. - Es compatible con MonsterASP free.

1.9 9. Sistema de emails (Gmail SMTP)

1.9.1 Configuración

En `Web.config` (local) y vía `secrets` de GitHub Actions (producción):

```
<appSettings>
  <add key="Smtp:Host" value="smtp.gmail.com" />
  <add key="Smtp:Port" value="587" />
  <add key="Smtp:User" value="alejandro.tu@gmail.com" />
  <add key="Smtp:Password" value="abcd efgh ijkl mnop" /> <!-- App Password -->
  <add key="Smtp:From" value="alejandro.tu@gmail.com" />
  <add key="App:BaseUrl" value="https://gestor-financiero.runasp.net" />
</appSettings>
```

1.9.2 Por qué App Password y no la contraseña normal

Desde mayo 2022, Google requiere **App Passwords** para SMTP: 1. Activar verificación en 2 pasos en la cuenta Google. 2. Generar una "App Password" en <https://myaccount.google.com/apppasswords> (16 caracteres). 3. Esa es la que va en SmtP:Password.

Si se usa la contraseña normal, Google bloquea el envío como "intento de acceso no seguro".

1.9.3 Plantillas HTML

App_Start/EmailService.cs tiene BuildOtpRegistroHtml(nombre, codigo) y BuildOtpResetHtml(nombre, codigo) que arman un correo profesional con: - Header con gradiente indigo y logo \$. - Saludo personalizado. - Bloque centrado con el código de 6 dígitos en monoespaciada gigante (38px) con border punteado. - Aclaración de expiración. - Footer institucional.

1.9.4 Envío asíncrono

```
public async Task SendAsync(string to, string subject, string htmlBody)
{
    using (var msg = new MailMessage())
    using (var client = new SmtPClient("smtp.gmail.com", 587))
    {
        client.EnableSsl = true;
        client.Credentials = new NetworkCredential(user, password);
        await client.SendMailAsync(msg);
    }
}
```

async/await evita bloquear el thread mientras se conecta a Gmail (que puede tardar 1-3 seg).

1.10 10. Configuración local

1.10.1 Prerrequisitos

1. **Windows 10/11**
2. **Visual Studio 2019/2022** (Community es suficiente) con carga *ASP.NET* y *desarrollo web*.
3. **.NET Framework 4.7.2 Developer Pack**.
4. **Docker Desktop** corriendo (con WSL2 backend).
5. **Node.js 18+** con npm.

1.10.2 Pasos para correr el proyecto localmente

```
# 1. Clonar el repo
git clone https://github.com/<usuario>/gestorFinanciero.git
cd gestorFinanciero

# 2. Levantar SQL Server en Docker (crea contenedor, ejecuta init.sql)
.\start.ps1

# 3. Instalar dependencias frontend y compilar assets
npm install
npm run build

# 4. Aplicar migraciones a la BD local
```

```
.\migrate.bat

# 5. Abrir gestor_financiero.sln en Visual Studio
# 6. Build → Rebuild Solution (restaura paquetes NuGet)
# 7. F5 para correr

# La app abre en https://localhost:44369
```

1.10.3 Comandos útiles

```
.\start.ps1          # arranca BD + abre VS
.\stop.ps1           # detiene contenedores
.\stop.ps1 -Volumes  # detiene + borra datos (reset total)
.\migrate.ps1 -Status # ver qué migraciones están pendientes
.\migrate.ps1        # aplica las pendientes
npm run watch         # Tailwind en modo desarrollo (recompila al guardar)
```

1.11 11. Configuración del servidor MonsterASP

1.11.1 Recursos en MonsterASP (gratis)

Al crear cuenta en <https://www.monsterasp.net> obtén: - **1 site** (subdominio gratis: `tuapp.runasp.net` con HTTPS automático). - **1 base SQL Server** (150 MB). - **Soporte .NET Framework 4.x**. - **FTP** para subir archivos. - **Panel web** (myLittleAdmin) para administrar la BD.

1.11.2 Configuración del site

Panel MonsterASP → **Add New Site**: - Subdomain: `gestor-financiero` → URL: `https://gestor-financiero.ru`
- ASP.NET version: 4.x

1.11.3 Configuración de la BD

Panel MonsterASP → **Add New Database**: - Type: MSSQL. - Name: definido al crear (ej. `db52546`). - Password: la que defines.

Te genera dos hostnames: - **Local access**: `db52546.databaseasp.net,1433` — usado por la app **desde dentro** del server de MonsterASP. - **Public access**: `db52546.public.databaseasp.net,1433` — usado **desde fuera** (ej. SSMS desde tu PC, GitHub Actions).

En `Web.Release.config` se usa el “Local access”. En el workflow de migraciones se usa el “Public access”.

1.11.4 Inicialización de la BD remota

Una vez creada la BD vacía, hay que crear las tablas. Dos opciones:

Opción A: conectarte con SSMS desde tu PC al “Public access” y ejecutar `database/init-cloud.sql`.

Opción B: panel MonsterASP → **Manage Database** → myLittleAdmin → Tools → SQL Query → pegar y ejecutar.

`init-cloud.sql` es idéntico a `init.sql` pero **sin** `CREATE DATABASE` ni `USE` (la BD ya existe).

1.12 12. CI/CD con GitHub Actions

1.12.1 Flujo del deploy

Cuando se hace git push a main o master, GitHub Actions ejecuta el workflow .github/workflows/deploy.yml:

1. Checkout código
2. Setup Node.js 20
3. npm install + npm run build (compila Tailwind, copia JS libs)
4. Setup MSBuild + NuGet restore
5. Inyectar credenciales en Web.Release.config
 - └ Reemplaza __DB_HOST__, __DB_NAME__, etc.
 - └ Reemplaza __SMTP_USER__, __SMTP_PASSWORD__, etc.
6. Aplicar migraciones SQL a la BD remota
 - └ Crear tabla _AppliedMigrations si no existe
 - └ Para cada migration *.sql:
 - └ Si ya está en _AppliedMigrations → salta
 - └ Si no → ejecuta + registra
 - └ Si falla → ABORTA el deploy (no sube archivos)
7. MSBuild compila en Release → carpeta publish/
8. Copiar assets npm (CSS, fuentes, JS) a publish/
9. FTP-Deploy sube publish/ a MonsterASP /wwwroot

1.12.2 Secrets de GitHub

Configurados en Settings → Secrets and variables → Actions:

Secret	Para qué
DB_HOST	Connection string de la app (local access)
DB_HOST_REMOTE	Conexión de GitHub Actions a la BD (public access)
DB_NAME, DB_USER, DB_PASSWORD	Credenciales BD
FTP_HOST, FTP_USER, FTP_PASSWORD	Credenciales FTP MonsterASP
SMTP_USER, SMTP_PASSWORD, SMTP_FROM	Credenciales Gmail SMTP
APP_BASE_URL	URL pública del sitio para los emails

Los secrets NO se commitean al repo. GitHub los inyecta en runtime como variables de entorno.

1.12.3 Sistema de migraciones SQL

Filosofía: **migraciones versionadas e idempotentes**.

Versionadas: cada cambio al schema queda en un archivo database/migration_YYYY-MM-DD_descripcion.sql. Quedan en git, ordenadas alfabéticamente.

Idempotentes: cada script usa IF NOT EXISTS / IF EXISTS para que pueda correr múltiples veces sin romper:

```
IF NOT EXISTS (SELECT 1 FROM sys.columns
               WHERE Name = 'activo' AND Object_ID = Object_ID('Usuario'))
  ALTER TABLE Usuario ADD activo BIT NOT NULL DEFAULT 1;
```

Tracking: tabla _AppliedMigrations(nombre PK, aplicada_en) registra cada migración exitosa. El workflow consulta esta tabla antes de aplicar cada script — si ya está, lo salta. Patrón similar al de Flyway / Liquibase / EF Migrations.

1.12.4 Script local de migraciones

migrate.ps1 (con migrate.bat como wrapper) hace lo mismo que el workflow pero contra la BD local en Docker. Se usa así:

```
.\migrate.bat -Status # ver pendientes (sin aplicar)
.\migrate.bat        # aplicar pendientes
```

1.13 13. Posibles preguntas del docente

1.13.1 “¿Por qué eligieron este stack?”

Combinación de **economía** (hosting gratis solo soporta .NET Framework), **productividad** (Razor + EF + Tailwind permite hacer mucho rápido) y **madurez** (las tecnologías tienen 10+ años, hay documentación y comunidad). Si el proyecto creciera y hubiera presupuesto, migraríamos a .NET 8 + Azure.

1.13.2 “¿Cómo manejan las contraseñas?”

Las hasheadamos con **PBKDF2** usando System.Web.Helpers.Crypto. 1000 iteraciones HMAC-SHA1 con salt aleatorio de 16 bytes por contraseña. La BD nunca guarda la contraseña en claro. Para verificar, comparamos el hash del input contra el almacenado.

1.13.3 “¿Cómo evitan que un usuario vea datos de otro?”

Tres capas: (1) Forms Authentication identifica al usuario mediante una cookie firmada. (2) [Authorize] global rechaza acceso sin autenticar. (3) En cada controller, todas las queries LINQ filtran por IdUsuario == CurrentUserId. Aun manipulando URLs (/Transaccion/Edit/123), el sistema devuelve 404 si el ID no pertenece al usuario.

1.13.4 “¿Qué pasa si la red entre la app y la BD se cae?”

Entity Framework abre una conexión SQL por request y la libera. Si falla la red, EF lanza una SqlException y el controller la deja propagar, mostrando una página de error. No hay degradación graceful: la app **necesita** la BD para funcionar (no hay cache local). Sí tenemos Connection Timeout=30 para que no falle si la BD está “fría”.

1.13.5 “¿Cómo escalarían si tuvieran 10,000 usuarios?”

Pasos en orden: (1) Migrar de hosting compartido gratis a un VPS / Azure App Service. (2) Separar BD a SQL Azure con réplicas de lectura. (3) Agregar Redis para cache de queries pesadas (resúmenes anuales, dashboards). (4) Migrar a .NET 8 que tiene mejor performance. (5) Si se vuelve crítico, romper el monolito en microservicios (auth, transacciones, reportes) — pero eso solo si la complejidad lo justifica.

1.13.6 “¿Qué pasa si dos usuarios intentan editar el mismo dato a la vez?”

Como cada dato pertenece a **un solo usuario** (es app personal, no colaborativa), el problema no se presenta. EF usa optimistic concurrency por default: si dos requests del mismo usuario llegan simultáneas, el segundo gana (último en escribir). No usamos [ConcurrencyCheck] porque no es necesario.

1.13.7 “¿Cómo se hace el deploy?”

Push a main → GitHub Actions corre el workflow → compila Tailwind y Chart.js, restaura NuGet, inyecta secrets en Web.Release.config, **aplica migraciones SQL pendientes a la BD remota**, compila el proyecto con MSBuild, sube los archivos por FTP a MonsterASP. Tarda ~3 minutos. Si la migración falla, aborta antes de subir, así nunca queda código nuevo con BD vieja.

1.13.8 “¿Cómo agregarían una nueva funcionalidad?”

1. Si requiere cambios a la BD, crear database/migration_YYYY-MM-DD_descripcion.sql idempotente.
2. Agregar/modificar modelos en Models/ y actualizar FinanzasContext si hay tablas nuevas.
3. Agregar/modificar el controller (heredando de BaseAuthController si es por-usuario).
4. Crear las vistas Razor en Views/<Controller>/.
5. Registrar las vistas y archivos .cs nuevos en gestor_financiero.csproj.
6. Correr .\migrate.bat para aplicar la migración local.
7. F5 y probar.
8. git commit && git push — el deploy automático corre la migración en prod y sube el código.

1.13.9 “¿Qué pasa si quieren cambiar de hosting?”

El proyecto está hecho para ser **portable**. Web.config y Web.Release.config tienen la connection string como variable. El workflow inyecta secrets. Cambiar de MonsterASP a, por ejemplo, Azure App Service implica: 1. Mover la BD (export + import del SQL). 2. Cambiar los secrets de GitHub (DB_HOST, FTP_* por las credenciales nuevas). 3. Reemplazar el step de FTP por el de Azure Web Deploy.

No habría que tocar código.

1.13.10 “¿Cómo prueban el envío de emails?”

Localmente: con un Gmail real + App Password configurada en Web.config. El usuario se registra con un email que él controla, recibe el código en su Gmail, lo ingresa. Funciona idéntico a producción. En la próxima versión consideraríamos un servicio tipo Mailtrap.io para tener un inbox de pruebas sin enviar a usuarios reales.

1.13.11 “¿Cómo recuperan datos si la BD se corrompe?”

Actualmente no tenemos backups automáticos en MonsterASP (limitación del plan free). Como mitigación: el código de migraciones es versionado, así que el schema se puede reconstruir desde cero ejecutando init.sql + todas las migraciones. Los datos sí se perderían — para producción real haríamos backups diarios con un script que use BACKUP DATABASE.

1.13.12 “¿Por qué SweetAlert y no alertas nativas del navegador?”

Las nativas (alert(), confirm()) son **modales bloqueantes** y feas. SweetAlert2 ofrece toasts no bloqueantes, animaciones, íconos, posicionables, autohide. Mucho mejor UX. Es una librería estándar de la industria (~50K stars en GitHub).

1.13.13 “¿Cómo decidieron qué reportes hacer?”

Pensamos en las 4 preguntas más comunes que se hace alguien que controla sus finanzas: (1) ¿en qué gasté el último mes? (Transacciones), (2) ¿cuánto debo y a quién? (Deudas), (3)

¿cumplí mi plan presupuestario? (Presupuesto), (4) ¿cómo evolucionó mi año? (Resumen Anual). Los 4 reportes responden esas preguntas con filtros y totales relevantes.

1.13.14 “¿Es segura su autenticación?”

Es razonablemente segura para una app no-financiera-crítica. Tenemos: PBKDF2 para passwords, OTP de 6 dígitos para verificar email, anti-forgery tokens en forms, cookies HttpOnly, machineKey fija. **No** tenemos: 2FA opcional con TOTP (solo el OTP por email), rate limiting en login, lockout tras N intentos fallidos. Para un sistema bancario real agregaríamos eso. Para gestión personal está OK.

1.14 14. Limitaciones conocidas

- **Plan free de MonsterASP:** 150 MB de BD, hosting compartido (cold starts ocasionales).
 - **Gmail SMTP:** límite informal de 500 emails/día. Si el sistema creciera, migraríamos a SendGrid o similar.
 - **PDFs son imagen:** el texto del PDF generado por html2pdf no es seleccionable. Para PDFs nativos habría que usar PdfSharp o similar (más trabajo).
 - **No hay rate limiting:** alguien podría intentar muchísimos logins. Mitigación a futuro: con [AllowAnonymous] y un sliding window en memoria.
 - **No hay 2FA opcional:** solo el OTP por email para registro/recovery, pero no para login regular.
 - **No multilinguaje:** todo el texto está en español hardcodeado. Para internacionalización habría que usar Resources/.resx.
-

1.15 15. Resumen ejecutivo en una página

Gestor Financiero es una aplicación web ASP.NET MVC 5 (.NET Framework 4.7.2) con front en Razor + Tailwind CSS, base SQL Server 2022, autenticación Forms con PBKDF2, verificación por OTP vía Gmail SMTP, y exportación de reportes a PDF en cliente con html2pdf.js. Está desplegada gratis en MonsterASP.NET, con CI/CD automático mediante GitHub Actions que compila, aplica migraciones SQL y sube vía FTP. La arquitectura es monolítica con patrón MVC; las consultas a la BD usan LINQ via Entity Framework 6, traducido automáticamente a SQL parametrizado. Cada usuario solo ve sus propios datos gracias a un filtro [Authorize] global combinado con verificación de ownership en cada query (Where (x => x.IdUsuario == CurrentUserId)).

Fecha de la documentación: Mayo 2026 **Versión del documento:** 1.0